0.6 UML 2.0

0.6.1 Einleitung

0.6.1.1 UML – Unified Modeling Language

UML-Standard

- Internationaler Standard der OMG (Open Management Group)
- Aktueller Standard (Seit März 2005): UML 2.0

Anwendungsgebiete

- Modellierung
- Dokumentation
- Spezifizierung
- Visualisierung

komplexer Softwaresysteme

Was UML *nicht* ist:

- Programmiersprache
- rein formale Sprache
- spezialisiert auf bestimmtes Anwendungsgebiet
- vollständig

Forward Engineering

- *Erzeugen* von Programmcode aus UML-Modell
- *Direkt möglich*, da Abbildungen von UML zu Programmiersprachen existieren

Reverse Engineering

- *Erzeugen* von UML-Modell aus Programmcode
- Nur möglich durch Unterstützung eines Programmierers

© R.-R. Grigat 2003-2019

- Programmcode enthält i. A. weniger Information als UML-Modell

Round-Trip Engineering

- Verbindung von Forward- und Reverse-Engineering
- UML-Tools stellen Konsistenz zwischen Code und Modell sicher

0.6.1.2 Diagrammtypen: „Es gibt mehr als nur Klassendiagramme“

6 *Struktur*diagramme

1. Klassendiagramm (im folgenden näher erläutert)
   - Beschreibt *statische* Struktur des Systems
2. Paketdiagramm
   - *Logische* Zusammenfassung von Modellelementen
   - Modellierung von Abhängigkeiten und Inklusion
3. Objektdiagramm
   - Zeigt beispielhaft Objekte und Attributbelegungen zu bestimmtem *Zeitpunkt*
4. Kompositionsstrukturdiagramm (Architekturdiagramm)
   - *Top-down*-Modellierung des Systems
5. Komponentendiagramm
   - Zeigt Organisation und Abhängigkeiten einzelner technischer Systemkomponenten
   - Modellierung der *Schnittstellen*
6. Verteilungsdiagramm
   - Zeigt Laufzeitumfeld (z. B. Hardware, Server) des Systems
   - Hohes *Abstraktionsniveau*

4 *Verhaltens*diagramme

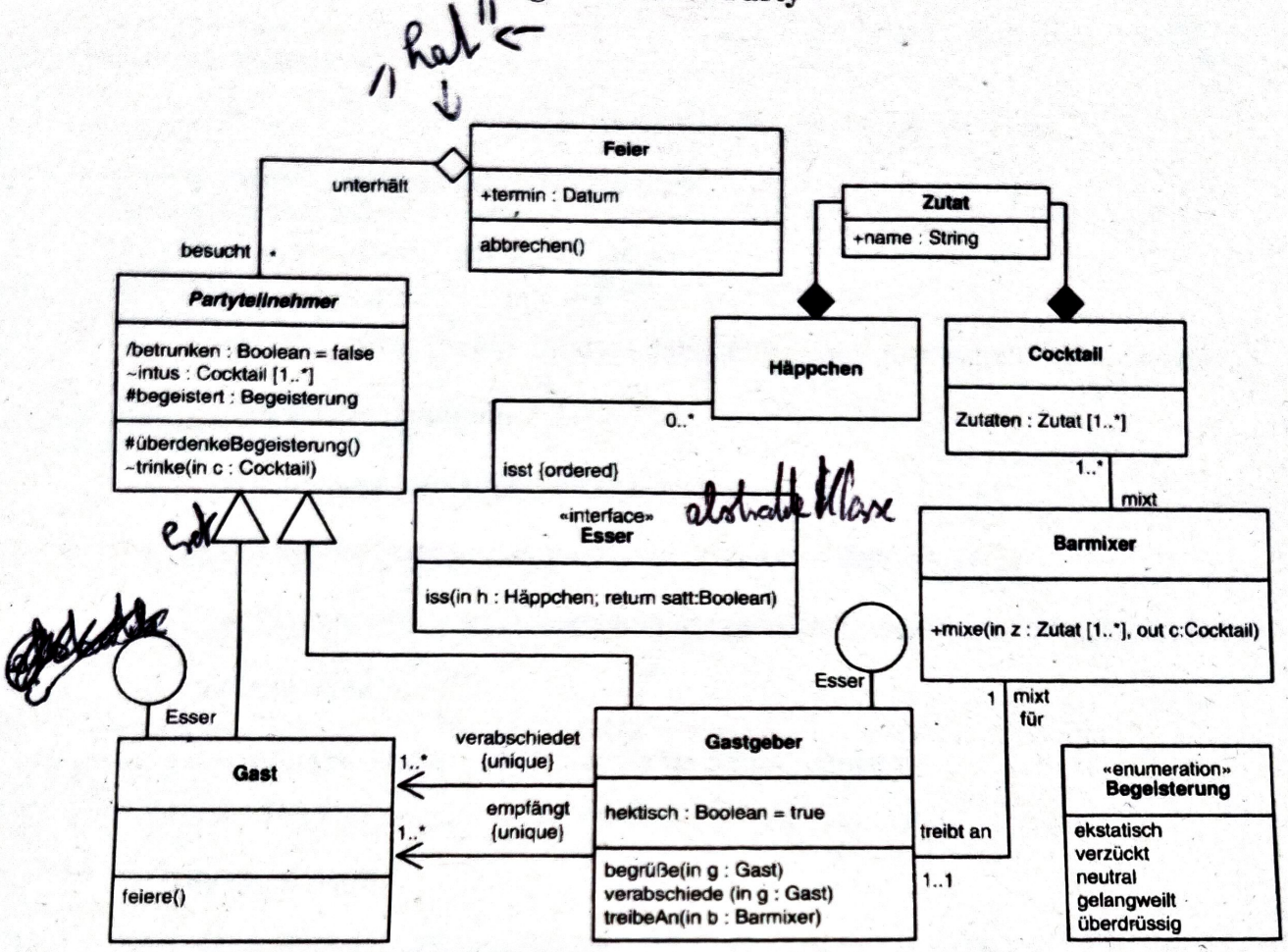
© R.-R. Grigat 2003-2019

1. Use-Case-Diagramm
   - Außenansicht auf das Gesamtsystem
2. Aktivitätsdiagramm
   - Detaillierte Visualisierung von Abläufen
   - Verwandschaft zu Petri-Netzen
3. Zustandsautomat
   - Zustandsmodell mit Zuständen, Ereignissen, Nebenläufigkeiten, Bedingungen, Ein- und Austrittsaktionen
4. Interaktionsdiagramme
   - Sequenzdiagramm
     - Informationsaustausch zwischen Kommunikationspartnern
     - *Detaillierte* Darstellung der zeitlichen Abfolge
   - Kommunikationsdiagramm
     - Informationsaustausch zwischen Kommunikationspartnern
     - *Überblick*
   - Timing-Diagramm
     - Exaktes *zeitliches* Verhalten von Klassen, Schnittstellen, ...
   - Interaktionsübersichtsdiagramm
     - Verbindet Interaktionsdiagramme
     - Hohes Abstraktionsniveau

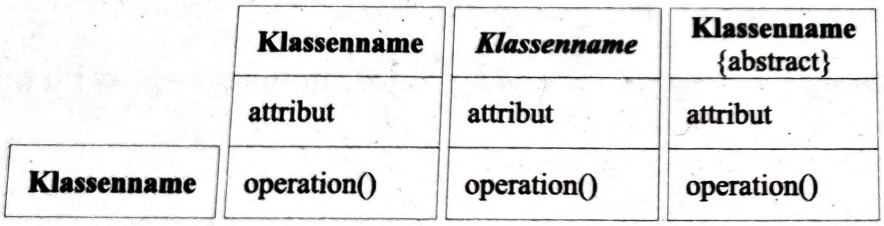
0.6.2 Klassendiagramme

© R.-R. Grigat 2003-2019

0.6.2.1 Beispiel: Klassendiagramm einer Party



0.6.2.2 Klassen



(a) einfache Klasse

(b) Klasse mit Attribut und Operation

(c) Abstrakte Klasse

(d) Abstrakte Klasse, alternative Darstellung

Abbildung 24: Darstellungen einer Klasse

© R.-R. Grigat 2003-2019

Klasse

- Menge von gleichartigen Objekten
- Gemeinsame Semantik
- Gemeinsame Eigenschaften (*Attribute*)
- Gemeinsames Verhalten

0.6.2.3 Attribut (in C++ / Java: Membervariable)

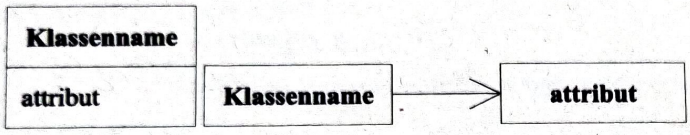


Abbildung 25: Darstellungsformen eines Attributs

Attributdeklaration

```
[Sichtbarkeit] [/] name [: Typ] ['['Multiplizität']']
  [= Vorgabewert] [{Eigenschaftswert}]
```

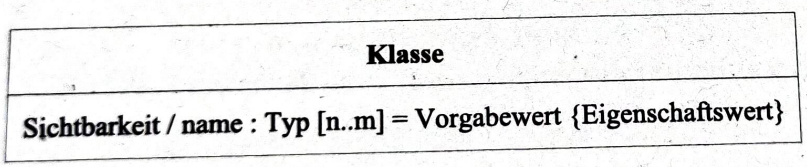


Abbildung 26: Klasse mit allgemeinem Attribut

Sichtbarkeit	Symbol	Bedeutung
public	+	Zugriff von allen Systemkomponenten erlaubt
private	-	Zugriff nur innerhalb von Ausprägungen der Klasse erlaubt
protected	#	Zugriff von Ausprägungen der Klasse und Spezialisierungen erlaubt
package	~	Zugriff von Klassen desselben Pakets erlaubt

Tabelle 7: Sichtbarkeiten

© R.-R. Grigat 2003-2019

Bedeutung von /

- Zur Laufzeit berechnetes Attribut
- Nicht abgespeichert
- Beispiel: Alter aus Geburtsdatum und Systemzeit errechnen

name

- Attributname
- Beliebige Zeichen
- Leer- und Sonderzeichen in UML erlaubt, sollten aber für Umsetzung in Programmiersprache *(forward engineering)* vermieden werden
- Mit *Kleinbuchstaben* beginnen
- Üblicherweise *camelCaseNotation*

Typ

- Datentyp des Attributs
- Beliebige zur Verfügung stehende Typen (auch andere Klassen) möglich

Multiplizität

- Anzahl unter dem Namen ablegbarer Instanzen
- Jede beliebige natürliche Zahl erlaubt
- Trennung von Unter- und Obergrenze durch `..`
- `*` steht für beliebige Anzahl
- Vorgabewert: `[1..1]`
- `[1..n]` kann auch als `[n]` geschrieben werden

Vorgabewert

- Attribut wird zu Beginn automatisch auf angegebenen Wert gesetzt

© R.-R. Grigat 2003-2019

Eigenschaftswerte (Auswahl)

readOnly Wert darf nicht verändert werden

ordered Inhalte geordnet, ohne Duplikate

bag Inhalte ungeordnet, Duplikate erlaubt

sequence Inhalte geordnet, Duplikate erlaubt

composite Attribut verantwortlich für Zerstörung der Inhalte

Selbstdefinierte Eigenschaftswerte sind zulässig.

Klassenattribut

- Alle Instanzen einer Klasse teilen sich denselben Speicherplatz
- Bezeichnung in einigen Programmiersprachen: *statisch*
- Darstellung durch Unterstreichung

Beispiel

BeispielKlasse
a : int #b : bool +pi : double = 3.14 {readOnly} -c : string[0..*] {sequence} d : int {d<5}

Abbildung 27: Klasse mit Attributen

a Attribut vom Typ `int`, Sichtbarkeit nicht spezifiziert

b Attribut vom Typ `boolean`, protected

pi Statische Konstante vom Typ **double**

c Privat, geordnete Menge, Duplikate zulässig

d Attribut vom Typ `int` mit Einschränkung

© R.-R. Grigat 2003-2019

0.6.2.4 Operation (in C++: Memberfunktion, in Java: Methode)

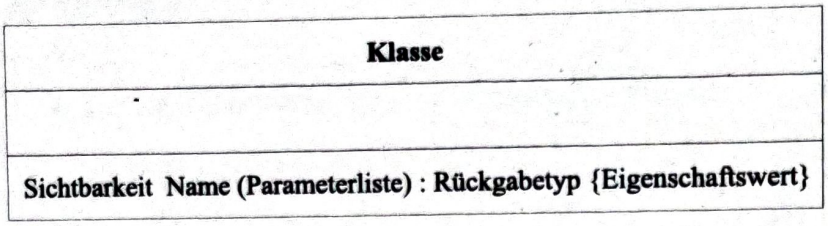


Abbildung 28: Darstellung einer Operation

Operationsspezifikation

```
[Sichtbarkeit] Name (Parameterliste) : Rückgabetyp [{Eigenschaftswert}]
```

Parameterliste

```
[Übergaberichtung] Name : Typ ['['Multiplizität ']']
    [= Vorgabewert] [{Eigenschaftswert}]
```

Sichtbarkeit

public +

private -

protected #

package ~

Name

- Benennung der Operation
- Verben verwenden:
  - Infinitiv (`anzeigen()`)
  - Imperativ (`zeigeAn()`)

Parameterliste

- Übergaberichtung

© R.-R. Grigat 2003-2019

in Parameter wird nur eingelesen

out Parameter wird nur ausgegeben

inout Parameter wird eingelesen und ausgegeben

return Rückgabeparameter

Rückgabetyp

- Datentyp (skalarer Typ oder Klasse) des Rückgabewertes

Eigenschaftswert

- Angabe zusätzlicher Eigenschaften (wie bei Attributen)

Klassenoperation

- Methoden werden direkt auf Klasse ausgeführt, nicht auf Instanzen der Klasse
- Darstellung durch Unterstreichung

Beispiel

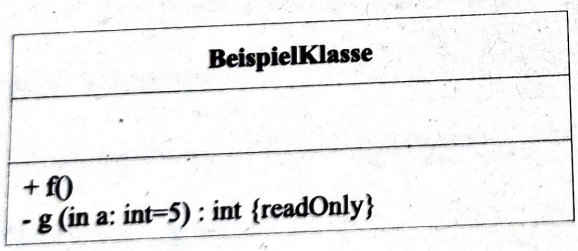


Abbildung 29: Klasse mit Operationen

f kein Übergabe- oder Rückgabeparameter, public

g Übergabeparameter vom Typ `int` (lesend) Ohne Vorgabe wird Belegung 5 angenommen Rückgabewert vom Typ `int` `readOnly`, d. h. Systemzustand bleibt konstant private

© R.-R. Grigat 2003-2019

0.6.2.5 Schnittstelle (in Java: Interface)

Abbildung 30: Darstellung einer Schnittstelle

Schnittstelle

- Abstrakte Klasse
- Instanzen (Objekte) sind nicht erlaubt
- Konkrete Klassen *realisieren* Schnittstelle
- Realisierung ist besondere Form der Spezialisierung

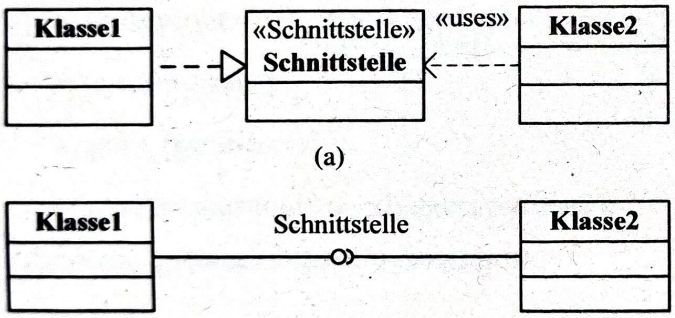


Abbildung 31: Implementierungs- (auch: Realisierungs-)beziehung (alternative Darstellungen)

0.6.2.6 Parametrisierte Klasse

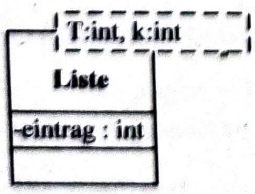


Abbildung 32: Beispiel für eine parametrisierte Klasse

© R.-R. Grigat 2003-2019

Parametrisierte Klassen

- Templates in C++
- Generics in Java

0.6.2.7 Generalisierung / Spezialisierung (In C++ / Java: Vererbung)

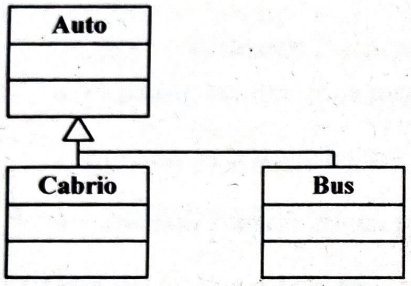


Abbildung 33: Darstellung von Generalisierung (Spezialisierung)

Generalisierung / Spezialisierung

- `Auto` ist *Generalisierung* von `Cabrio`
- `Cabrio`, `Bus` sind *Spezialisierungen* von `Auto`

Vererbung / Polymorphie

- *Vererbung*: Eigenschaften (Attribute, Assoziationen, ...) von `Auto` gehen über auf `Cabrio`, `Bus`
- *Polymorphie*: Instanz von `Auto` kann durch Instanzen von `Cabrio`, `Bus` *substituiert* werden („ist-ein"-Beziehung)

Generalisierungsmenge

- Menge der Spezialisierungen, gemeinsame Generalisierung
- Beispiel: Generalisierungsmenge Fahrzeugtypen
- *Partitionen*: Die einzelnen Spezialisierungen (`Cabrio`, `Bus`)

© R.-R. Grigat 2003-2019

Generalsierungseigenschaften

(in-)complete (Nicht) alle sinnvollen Spezialisierungen erwähnt

disjoint Instanzen gehören eindeutig zu *einem* Subtypen Beispiel: `Cabrio` ist nicht auch `Bus`

overlapping Inversion von `disjoint`

0.6.2.8 Assoziation

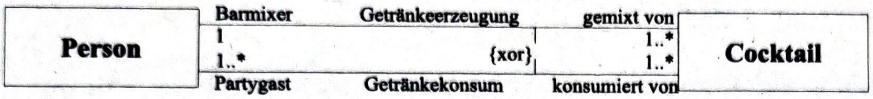
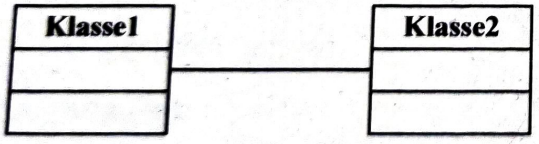


Abbildung 34: Beispiel für Assoziationen

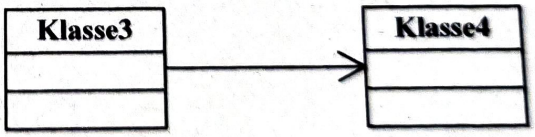
Optionale Elemente einer Assoziation

- *Assoziationsname* (Getränkeerzeugung)
- *Rolle* (Barmixer)
- *Multiplizität* (1)
- *Einschränkung* ({XOR}) (Barmixer kann nicht Partygast sein)
- *Eigenschaften* (ähnlich wie bei Attributen)

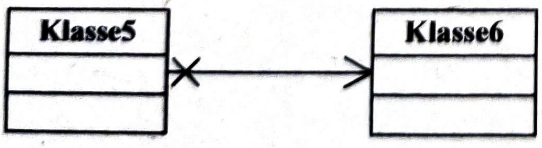
0.6.2.9 Navigierbare Assoziation



(a) unspezifiziert



(b) Klasse3 kennt Klasse4, umgekehrt unspezifiziert

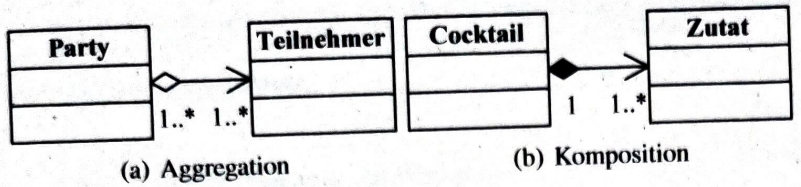


(c) Klasse5 kennt Klasse6, Klasse6 kennt Klasse5 nicht

Abbildung 35: Navigierbarkeit von Assoziationen

© R.-R. Grigat 2003-2019

0.6.2.10 Aggregation und Komposition



(a) Aggregation (b) Komposition

Abbildung 36: Aggregation und Komposition

Aggregation

- „Teil-Ganzes-Beziehung“
- Ganzes „*besteht-aus*“ Teilen
- Aggregiertes Objekt kann Teil mehrerer Objekte sein
- Lebensdauer beteiligter Objekte unabhängig voneinander

***Kompositionsaggregation* (kurz: *Komposition*)**

- Drückt physische Inklusion aus
- Teile und Ganzes bilden Einheit
- Ein Teil ist jeweils höchstens einem Ganzen zugeordnet
- Lebensdauer des Teils hängt von der des Ganzen ab

0.6.2.11 Abhängigkeit

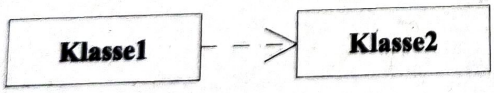


Abbildung 37: Darstellung einer Abhängigkeitsbeziehung

Abhängigkeitsbeziehung

- Darstellung durch gestrichelten Pfeil

© R.-R. Grigat 2003-2019

- Dient nur der Dokumentation
- Wird nicht in Programmiersprache übersetzt
- Element benötigt ein anderes, ohne dass zur Laufzeit eine Beziehung besteht
- Andernfalls Assoziation verwenden

0.6.2.12 Kommentar

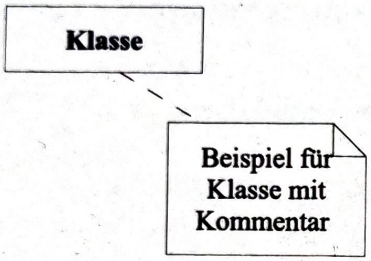


Abbildung 38: Darstellung eines Kommentars

© R.-R. Grigat 2003-2019

0.7 Entwurfsmuster (Design Patterns)

0.7.1 Einleitung

0.7.1.1 Literatur zu Entwurfsmustern

Literatur

- Erich Gamma, Richard Helm, Ralph Johnsons, John Vlissides
  „Design Patterns: Elements of Reusable Object-Oriented Software“
  deutsche Version:
  „Entwurfsmuster: Elemente wiederverwendbarer objektorientierter Software“

0.7.1.2 Entwurfsmuster

Definition: Entwurfsmuster
Beschreibung zusammenarbeitender Objekte und Klassen, die maßgeschneidert sind, um ein allgemeines Entwurfsproblem in einem bestimmten Kontext zu lösen

Ziele:

- Lösung für Klasse von Problemen beschreiben
- Ermöglichen, bei konkreter Programmieraufgabe auf bereits bekannte Entwurfsmuster zurückzugreifen
- Vermeiden, „das Rad neu zu erfinden“

0.7.1.3 Das Buch „Entwurfsmuster“

Das Buch von Gamma, Helm, Johnson, Vlissides

- Standardwerk zu Softwaredesign
- Beschreibt 23 Entwurfsmuster
- Ist als Katalog gedacht, um für konkrete Aufgaben passende Entwurfsmuster zu finden und anzuwenden
- Jedes Muster besteht aus vier Elementen:

© R.-R. Grigat 2003-2019

1. Musternamen
   - Standardisierter Name ermöglicht Diskussion über Softwareentwurf auf höherer Abstraktionsebene
2. Problemabschnitt
   - beschreibt, wann ein Muster anzuwenden ist
3. Lösungsabschnitt
   - bietet abstrakte Beschreibung mit Hilfe von Klassen und Objekten
4. Konsequenzenabschnitt
   - stellt Vor- und Nachteile eines Entwurfsmusters gegenüber

0.7.1.4 Einteilung von Entwurfsmustern

Einteilung nach Aufgabe

- Erzeugungsmuster
  - beschreiben, wie ein Objekt erzeugt werden kann, unabhängig von konkreten Datentypen
  - Beispiel: *Singleton*, erlaubt die Existenz von maximal einem Objekt dieser Klasse
- Strukturmuster
  - beschreiben, wie Klassen bzw. Objekte miteinander verknüpft werden können
  - Beispiel: *Adapter (Wrapper)* passt eine Schnittstelle an eine von einem anderen Objekt erwartete Schnittstelle an
  - Beispiel: *Proxy* kontrolliert Zugriff auf ein Objekt mit Hilfe eines vorgelagerten Objekts
- Verhaltensmuster
  - kapseln Algorithmen bzw. befassen sich mit der Interaktion von Objekten
  - Beispiel: *Iterator* greift auf Elemente eines zusammengesetzten Objekts sequentiell zu
  - Beispiel: *Strategie* definiert eine Familie von Algorithmen, die gekapselt und austauschbar sind

© R.-R. Grigat 2003-2019

– Beispiel: *Beobachter* definiert eine 1-zu-n-Abhängigkeit zwischen Objekten, so dass bei Änderung eines Objekts alle anderen abhängigen Objekte benachrichtigt und aktualisiert werden

0.7.2 Singleton

0.7.2.1 Singleton: UML-Klassendiagramm

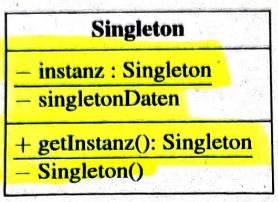


Abbildung 39: UML-Klassendiagramm: Singleton

Singleton-Muster

- ein Erzeugungsmuster
- ermöglicht genau eine Instanz aus der Klasse
- Konstruktor ist nicht öffentlich
- Instanz wird in privater statischer Variable gespeichert
- mittels öffentlicher statischer Memberfunktion wird Referenz auf Instanz geliefert

0.7.2.2 Beispiel: Singleton

Beispiel C++: Nur ein Auto erlauben

© R.-R. Grigat 2003-2019

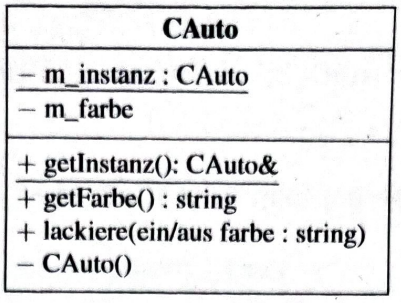


Abbildung 40: UML-Klassendiagramm: Singleton-Beispiel CAuto

```
1  //cauto.hpp
2  class CAuto
3  {
4    public:
5      static CAuto& getInstanz();
6      std::string getFarbe();
7      void lackiere(std::string farbe);
8    private:
9      CAuto();
10     CAuto(const CAuto& other);
11       // Kopierkonstruktor verboten
12     CAuto operator=(CAuto& other);
13       // Zuweisungsoperator verboten
14     static CAuto m_instanz;
15     std::string m_farbe;
16 };
```

- Verhindern, dass Default-Kopierkonstruktor und -Zuweisungsoperator von Compiler erstellt werden
- Dazu eigenen Kopierkonstruktor und Zuweisungsoperator deklarieren, ohne Definition zu erstellen

Beispiel C++: auto.cpp

```
1  //cauto.cpp
2  CAuto::CAuto():
3    m_farbe("rot")
4  {}
5  CAuto& CAuto::getInstanz()
```

© R.-R. Grigat 2003-2019

```
6 {
7   return m_instanz;
8 }
9 CAuto CAuto::m_instanz;
10   // Aufruf des Konstruktors
11
12 string CAuto::getFarbe()
13 {
14   return m_farbe;
15 }
16 void CAuto::lackiere(string farbe)
17 {
18   m_farbe = farbe;
19 }
```

- Erzeugung der (einzigen!) Instanz von `CAuto`
  - bei Beginn der Programmausführung
  - durch Initialisierung der statischen Membervariablen `m_instanz`
  - im statischen Speicher
- statische Methode `getInstanz()` liefert Referenz auf Instanz

Beispiel C++: Anwendung

```
1 //main.cpp
2
3   CAuto& dasAuto = CAuto::getInstanz();
4
5   //CAuto zweitwagen(dasAuto); // Compilerfehler
6
7   dasAuto.lackiere("gelb");
8   cout << dasAuto.getFarbe() << endl;
```

- `dasAuto` ist Referenz auf einzige Instanz
- eine Referenz auf diese Instanz liefert die statische Methode `CAuto::getInstanz()`
- Membermethoden `lackiere` oder `farbe` können ganz normal auf Instanz des Objekts angewandt werden
- Aufruf von Kopierkonstruktor oder Zuweisungsoperator würde zu Compilerfehler führen, da keine Implementierung vorhanden ist

© R.-R. Grigat 2003-2019

0.7.3 Adapter (Wrapper)

0.7.3.1 Adapter (Wrapper): UML-KLassendiagramm

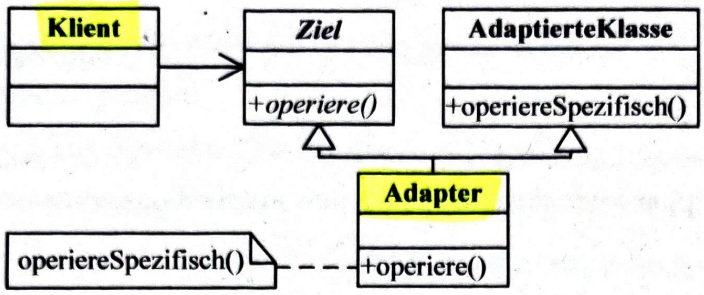


Abbildung 41: UML-Klassendiagramm: Adapter

Adapter-Muster (häufig auch Wrapper genannt)

- Ein Strukturmuster
- Ermöglicht die Verwendung einer Schnittstelle, die nicht der benötigten Schnittstelle entspricht
- **Adapter** erbt von der von **Klient** verwendeten Schnittstelle und von der zu adaptierenden Schnittstelle
- Der Aufruf einer **Operation** des Adapters bewirkt den Aufruf einer **spezifischen Operation** der adaptierten Klasse

0.7.3.2 Beispiel zu Adapter

Beispiel C++

- Zugekaufte Bibliothek enthält Klasse `CTaxi`
- `CTaxi` enthält Methode `reinigen()`, die der Methode `waschen()` aus der Klasse `CAuto` entspricht.
- Quellcode von `CTaxi` ist nicht verfügbar, Schnittstelle kann nicht verändert werden

© R.-R. Grigat 2003-2019

```cpp
//ctaxi.hpp
class CTaxi
{
  public:
    CTaxi();
    ~CTaxi();
    void reinigen();
    double taxameter();
  private:
    double m_taxameter;
};
```

// soll waschen();

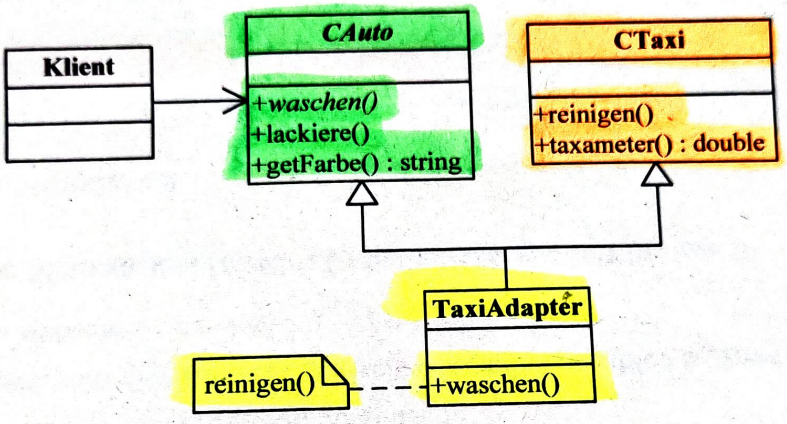


Abbildung 42: UML-Klassendiagramm: Adapter-Beispiel

Beispiel C++: Schnittstelle von CAuto

```cpp
//cauto.hpp
class CAuto
{
  public:
    CAuto();
    CAuto(std::string farbe);
    virtual ~CAuto();
    virtual std::string getFarbe();
    virtual void lackiere(std::string farbe);
    virtual void waschen() = 0;
  private:
    std::string m_farbe;
};
```

© R.-R. Grigat 2003-2019

- Abstrakte Klasse, `waschen()` ist rein virtuelle Funktion

Beispiel C++: Adapterklasse

```
1  //ctaxiadapter.hpp
2  class CTaxiAdapter :
3    public CAuto,
4    private CTaxi
5  {
6  public:
7    CTaxiAdapter();
8    ~CTaxiAdapter();
9    void waschen();
10 };
```

- **private** Vererbung von `CTaxi`
- Methoden von `CTaxi` sind nicht über `CTaxiAdapter` ansprechbar

```
1  //ctaxiadapter.cpp
2  CTaxiAdapter::CTaxiAdapter():
3    CAuto("taxigelb")
4  {}
5  CTaxiAdapter::~CTaxiAdapter()
6  {}
7  void CTaxiAdapter::waschen()
8  {
9    CTaxi::reinigen();
10 }
```

- Konstruktor von `CAuto` wird `"taxigelb"` übergeben
- `waschen()` ruft `reinigen()` aus `CTaxi` auf

Beispiel C++: Anwendung

```
1  //main.cpp
2    CAuto *taxi = new CTaxiAdapter;
3    taxi->waschen();
4    cout << taxi->getFarbe() << endl;
5    //cout << taxi->taxameter() << endl; // Compilerfehler
6    delete taxi;
```

//Polymorphie ("Upcast")

© R.-R. Grigat 2003-2019

- `CAuto*` kann Zeiger auf `CTaxiAdapter` enthalten, da `CAuto` Basisklasse ist
- `waschen()` ruft über den Adapter `reinigen()` auf
- `getFarbe()` ruft Methode aus `CAuto` auf
- Aufruf von `taxameter()` aus `CTaxi` ist nicht erlaubt
  - `CAuto` ist keine Basisklasse zu `CTaxi`
  - Außerdem ist Vererbung von `CTaxi` nach `CTaxiAdapter` privat
- **`delete`** nicht vergessen!

0.7.4 Proxy

0.7.4.1 Proxy: UML-Klassendiagramm

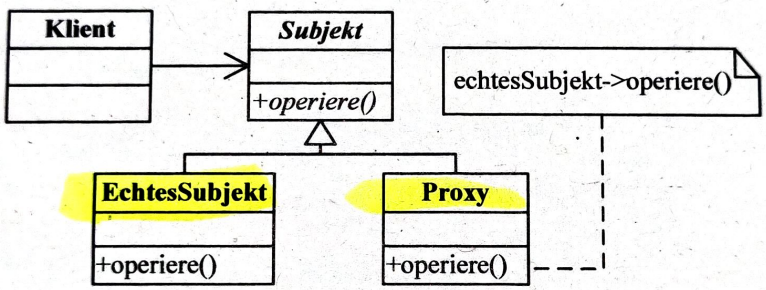


Abbildung 43: UML-Klassendiagramm: Proxy

Proxy-Muster

- kontrolliert Zugriff auf ein Objekt mit Hilfe eines vorgelagerten Objekts
- ermöglicht das Erzeugen von „teuren" Objekten (Rechenzeit, Speicherbedarf) erst auf Verlangen (lazy evaluation)
- erlaubt Smart-Pointer: Zeigervariablen, deren Speicherplatz automatisch freigegeben wird
- **Proxy** hat dieselbe Schnittstelle wie **EchtesSubjekt**
- **Klient** kann also EchtesSubjekt durch Proxy ersetzen
- **Klient** ruft Operation von **Proxy** auf
- **Proxy** steuert Ausführung der Operation von **EchtesSubjekt** (z. B. zeitverzögert)

© R.-R. Grigat 2003-2019

0.7.4.2 Beispiel zu Proxy

Beispiel C++: Smart Pointer für CAuto

- Im folgenden wird eine abgewandelte Variante des Proxy-Musters verwendet
- **Proxy** erbt hier nicht von **Subjekt**
- Operatoren -> (Zugriff) und * (Dereferenzierung) werden stattdessen überladen
- **Proxy** kann zwar nicht mehr in Variable vom Typ **Subjekt** gespeichert werden, **Klient** kann aber dennoch **Proxy** genau wie **Subjekt** verwenden

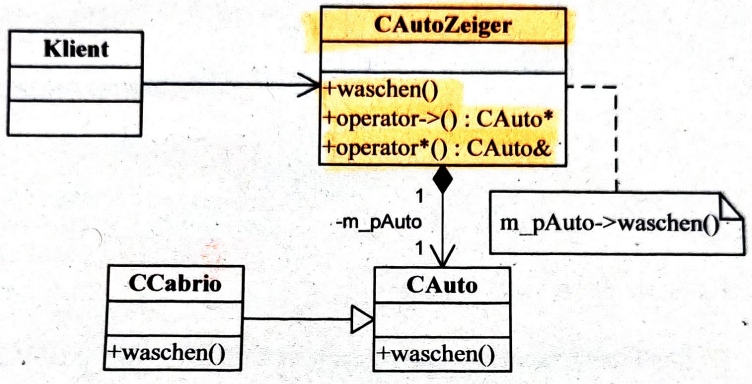


Abbildung 44: UML-Klassendiagramm: Proxy-Beispiel

- Statt Vererbung werden Operatoren -> (Zugriff) und * (Dereferenzierung) überladen
- `CAuto*` kann also durch `CAutoZeiger` ersetzt werden

```
1  //cautozeiger.hpp
2  class CAutoZeiger
3  {
4    public:
5      CAutoZeiger(CAuto *pauto);
6      ~CAutoZeiger();
7      CAuto* operator->();
8      CAuto& operator*();
9    private:
10     CAutoZeiger(const CAutoZeiger&);
11       //verboten
```

© R.-R. Grigat 2003-2019

```
12    CAutoZeiger operator=(CAutoZeiger);
13      //verboten
14
15    CAuto *m_pauto;
16 };
```

- eigentlicher Zeiger auf `CAuto` wird in `m_pauto` gespeichert
- **`operator->()`** überladen: führt `->` auf EchtesSubjekt aus
- **`operator*()`** überladen: liefert Inhalt von EchtesSubjekt als Referenz
- Kopien von Proxy verbieten!

Beispiel C++: Implementierung von `CAutoZeiger`

```
1  //cautozeiger.cpp
2  CAutoZeiger::CAutoZeiger(CAuto *pauto):
3    m_pauto(pauto)
4  {}
5  CAutoZeiger::~CAutoZeiger()
6  {
7    delete m_pauto;
8  }
9  CAuto* CAutoZeiger::operator->()
10 {
11   return m_pauto;
12 }
13 CAuto& CAutoZeiger::operator*()
14 {
15   return *m_pauto;
16 }
```

- *Konstruktor* von `CAutoZeiger` wird mit Zeiger auf ein Objekt vom Typ `CAuto` *initialisiert*
- Destruktor von `CAutoZeiger` ruft **`delete`** für das Objekt auf
- **`operator->()`** liefert Adresse des Objektes
- **`operator*()`** liefert Inhalt des Objektes

© R.-R. Grigat 2003-2019

Beispiel C++: Anwendung des Smart-Pointers

```
1 {
2   CAutoZeiger cabrio(new CCabrio());     // Aufrufe Konstruktoren
3   CAutoZeiger taxi(new CTaxiAdapter()); // CAutoZeiger(CAuto*)
4   cabrio->waschen();
5         //wird zu (cabrio.operator->())->waschen();
6         //zweites -> implizit durch Compiler eingefügt
7   taxi->waschen();
8 }
```

- explizites **delete** ist nicht erforderlich
- Objekte werden automatisch beim Erreichen von } gelöscht
  - Objekte vom `CAutoZeiger` werden im automatischen Speicher erzeugt
  - Daher wird Destruktor von `CAutoZeiger` am Ende des Gültigkeitsbereichs aufgerufen
  - Im Destruktor wird **delete** für das eigentliche Objekt aufgerufen

0.7.5 Strategie (Strategy)

0.7.5.1 Strategie (Strategy):UML-Klassendiagramm

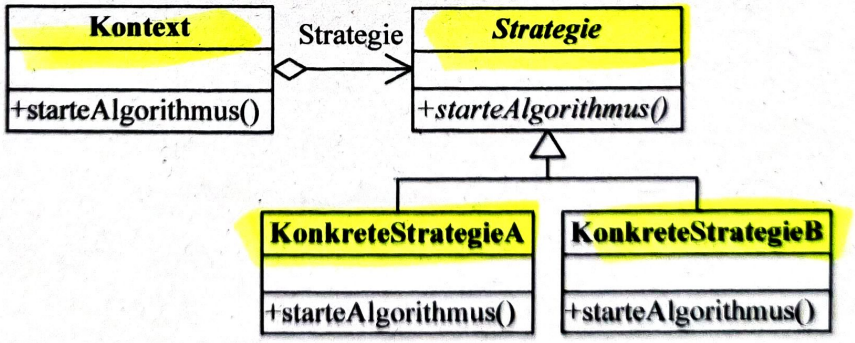


Abbildung 45: UML-Klassendiagramm: Strategie

Strategie-Muster

- Ein Verhaltensmuster
- Kapselt Algorithmen und macht sie austauschbar

© R.-R. Grigat 2003-2019

- **Strategie** deklariert Schnittstelle, die von allen Algorithmen angeboten wird
- **KonkreteStrategie** implementiert Algorithmus

0.7.5.2 Beispiel zu Strategie

Beispiel C++: Verschiedene Fahrstile

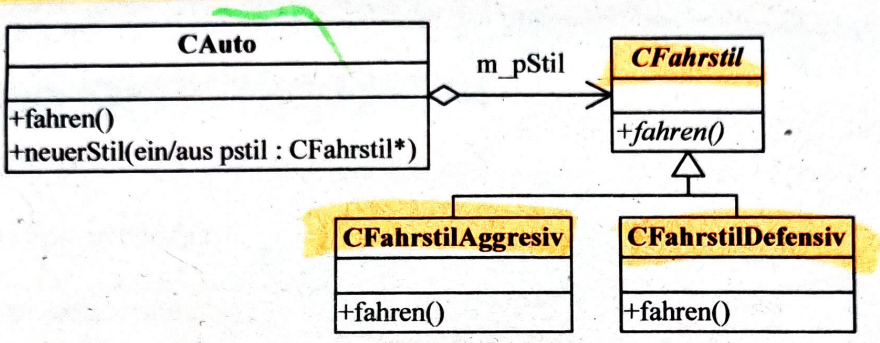


Abbildung 46: UML-Klassendiagramm: Strategie-Beispiel

- Abstrakte Klasse `CFahrstil` stellt Schnittstelle für Fahr-Algorithmen dar
- Abgeleitete Klassen implementieren Fahr-Algorithmen

```
1  //cfahrstil.hpp
2  class CFahrstil
3  {
4    public:
5      virtual void fahren() = 0;
6  };
7
8  //Abgeleitete Klasse, implementiert defensiven Fahr-Algorithmus
9  class CFahrstilDefensiv : public CFahrstil
10 {
11   public:
12     void fahren();
13 };
14
15 //Abgeleitete Klasse, implementiert aggressiven Fahr-Algorithmus
16 class CFahrstilAggressiv : public CFahrstil
17 {
18   public:
```

© R.-R. Grigat 2003-2019

```
19     void fahren();
20 };
```

© R.-R. Grigat 2003-2019

Beispiel C++: Implementierung verschiedener Algorithmen

```
1 //cfahrstil.cpp
2
3 //Abgeleitete Klasse, implementiert defensiven Fahr-Algorithmus
4 void CFahrstilDefensiv::fahren()
5 {
6   cout << "Laaangsaaam ... " << endl;
7 }
8
9 //Abgeleitete Klasse, implementiert aggressiven Fahr-Algorithmus
10 void CFahrstilAggressiv::fahren()
11 {
12   cout << "IDIOT!" << endl;
13 }
```

Beispiel C++: Klasse CAuto als Kontext

```
1 //cauto.hpp
2 class CAuto
3 {
4   public:
5     CAuto();
6     virtual ~CAuto();
7     virtual void neuerStil(CFahrstil* pStil);
8     virtual void fahren();
9   private:
10    CFahrstil* m_pStil;
11 };
```

- Zeiger auf aktuellen Fahrstil in `m_pStil`
- Fahrstil kann zur Laufzeit geändert werden über Methode `neuerStil(CFahrstil* pStil)`

Beispiel C++: Klasse CAuto als Kontext

```
1 //cauto.cpp
2 CAuto::CAuto():
3 m_pStil(new CFahrstilDefensiv)
4 {}
5 CAuto::~CAuto()
6 {
```

© R.-R. Grigat 2003-2019

```
7   delete m_pStil;
8 }
9 void CAuto::neuerStil(CFahrstil* pStil)
10 {
11   delete m_pStil;
12   m_pStil = pStil;
13 }
14 void CAuto::fahren()
15 {
16   m_pStil->fahren();
17 }
```

- Im Konstruktor wird mit **new** neuer Fahrstil erzeugt
- Dieser muss mit **delete** im Destruktor gelöscht werden
- Methode `neuerStil` erwartet Zeiger auf einen Stil, der außerhalb der Methode mit **new** erzeugt wurde
- Der alte Stil muss in der Methode `neuerStil` mit **delete** gelöscht werden.

Beispiel C++: Anwendung von Strategie

```
1  CCabrio cabrio;
2  CTaxiAdapter taxi;
3
4  cabrio.fahren();
5        // Ausgabe: Laaangsaaam ...
6  cabrio.neuerStil(new CFahrstilAggressiv);
7  cabrio.fahren();
8    // Ausgabe: IDIOT!
9  taxi.fahren();
10       // Ausgabe: Laaangsaaam ...
```

- Jedes Objekt kann seine eigene Strategie besitzen
- Strategie kann zu Laufzeit geändert werden

© R.-R. Grigat 2003-2019

0.7.6 Beobachter (Observer)

Beobachter-Muster

- Ein Verhaltensmuster
- Definiert 1-zu-n-Abhängigkeit zwischen Objekten
- Änderung des Zustands eines Objektes wird allen anderen beteiligten Objekten mitgeteilt

0.7.6.1 Beobachter: UML-Klassendiagramm

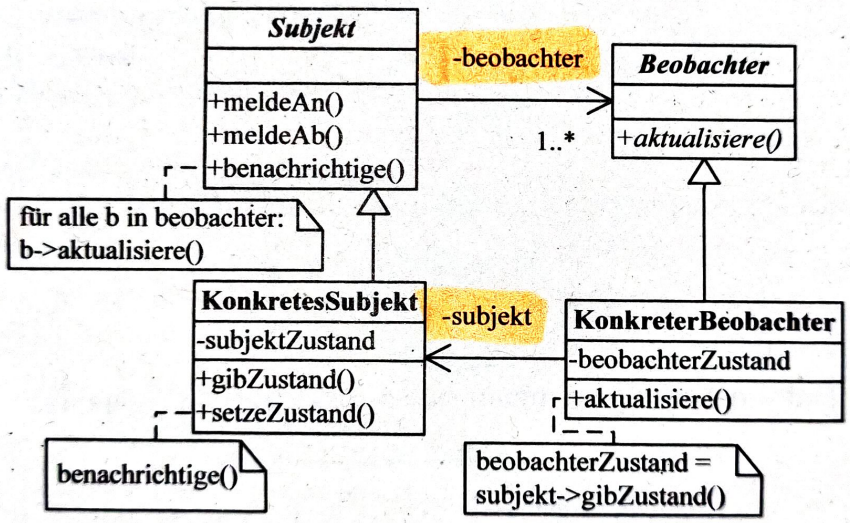


Abbildung 47: UML-Klassendiagramm: Beobachter

0.7.6.2 Teilnehmer an Beobachter-Muster

Subjekt

- Kennt seine Beobachter
- Wird von beliebig vielen Beobachtern beobachtet
- Bietet Schnittstelle zum An- und Abmelden von Beobachten

© R.-R. Grigat 2003-2019

Beobachter

- Definiert Aktualisierungsschnittstelle **aktualisiere()** für Objekte vom Typ **KonkreterBeobachter**, die über Änderungen eines Subjekts vom Typ **KonkretesSubjekt** benachrichtigt werden sollen

KonkretesSubjekt

- speichert den für KonkreterBeobachter relevanten Zustand **subjektZustand**
- benachrichtigt Beobachter, wenn sich Zustand ändert durch **benachrichtige()**

KonkreterBeobachter

- Verwaltet Referenz auf KonkretesSubjekt
- Speichert Zustand des Subjekts
- Implementiert Aktualisierungsschnittstelle

0.7.6.3 Beobachter: UML-Sequenzdiagramm

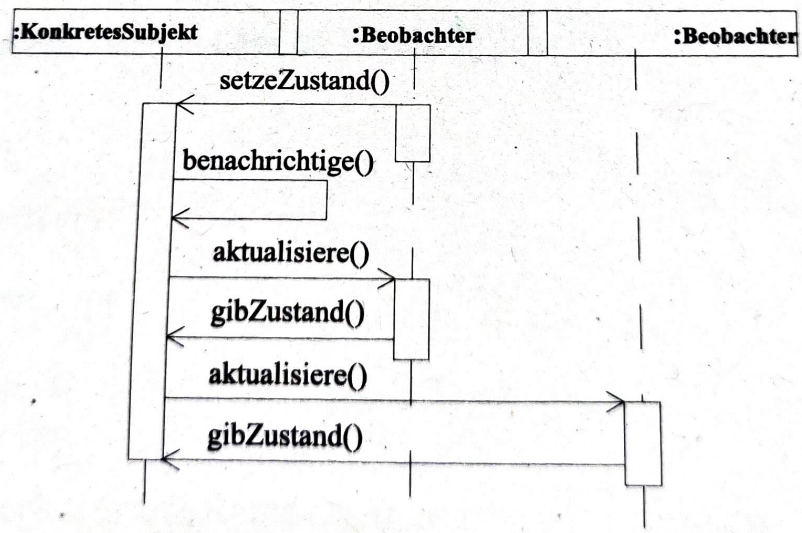


Abbildung 48: UML-Sequenzdiagramm: Beobachter

© R.-R. Grigat 2003-2019

0.7.6.4 Beispiel zu Beobachter: Autos beobachten Ampel

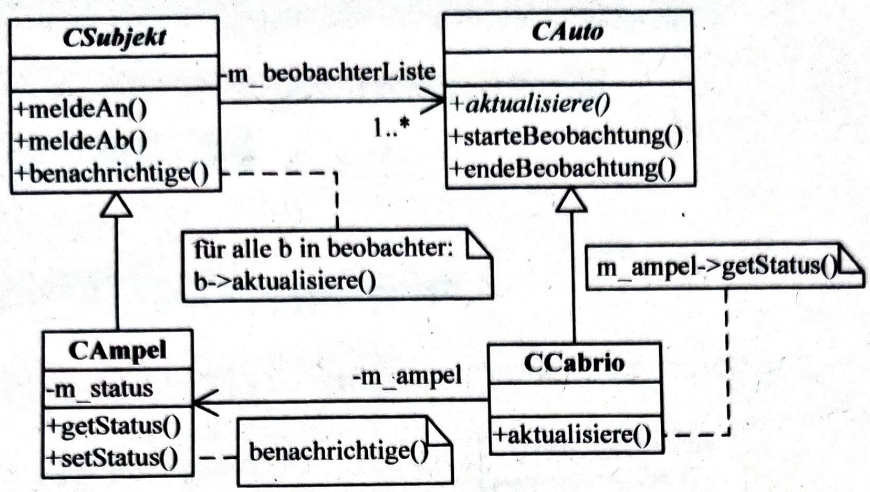


Abbildung 49: UML-Klassendiagramm: Beobachter-Beispiel

Beispiel C++: Beobachter

```
1  //cauto.hpp
2  class CAuto
3  {
4    public:
5      CAuto();
6      virtual ~CAuto();
7      virtual void aktualisiere() = 0; // rein virtuell
8      virtual void starteBeobachtung(CAmpel* ampel); // benötigen CSubjekt
9      virtual void endeBeobachtung();                // und CAmpel
10   protected:  // besser wäre private: sowie getValue(), setValue() Methoden
11     CAmpel* m_ampel;
12 };
```

- Abstrakte Klasse
- Bietet über `aktualisiere()` Schnittstelle zur Aktualisierung von beobachtenden Objekten (CAuto)
- `m_ampel` ist Zeiger auf Objekt, das beobachtet werden soll (KonkretesSubjekt bzw. CAmpel)

© R.-R. Grigat 2003-2019

Beispiel C++: Implementierung des Beobachters

```
1  //cauto.cpp
2  CAuto::CAuto() :
3  m_ampel(nullptr)
4  {}
5  CAuto::~CAuto()
6  {}
7  void CAuto::starteBeobachtung(CAmpel* ampel)
8  {
9    m_ampel = ampel;
10   m_ampel -> meldeAn(this);  // in beobachterListe eintragen
11 }
12 void CAuto::endeBeobachtung()
13 {
14   m_ampel -> meldeAb(this);  // aus beobachterListe löschen
15   m_ampel = nullptr;
16 }
```

- `meldeAn` fügt Zeiger auf aktuelles Objekt (`this`) der `beobachterListe` in `CSubjekt` hinzu
- `m_ampel` ist `nullptr`, wenn keine Ampel beobachtet wird

Beispiel C++: Subjekt

```
1  //csubjekt.hpp
2  class CAuto;// Ringförmige Abhängigkeit der Klassen in Beobachter-Muster,
3              // daher hier zusätzliche Deklaration von CAuto nötig
4  class CSubjekt
5  {
6    public:
7      virtual ~CSubjekt();
8      virtual void meldeAn(CAuto* beobachter); // benötigt CAuto, also
9      virtual void meldeAb(CAuto* beobachter); // ringförmige Abhängigkeit
10     virtual void benachrichtige();
11   protected:
12     CSubjekt();
13     private:
14       std::list<CAuto*> m_beobachterListe;
15 };
```

- Schnittstelle zum An- und Abmelden von Beobachtern

© R.-R. Grigat 2003-2019

- Beobachtende Objekte werden in `m_beobachterListe` gespeichert

Beispiel C++: Implementierung des Subjekts

```
1  // CSubjekt.cpp
2  CSubjekt::CSubjekt()
3  {}
4  CSubjekt::~CSubjekt()
5  {}
6  void CSubjekt::meldeAn(CAuto* beobachter)
7  {
8    m_beobachterListe.push_back(beobachter);
9  }
10 void CSubjekt::meldeAb(CAuto* beobachter)
11 {
12   m_beobachterListe.remove(beobachter);
13 }
14 void CSubjekt::benachrichtige()
15 {
16   list<CAuto*>::iterator pos;
17   for (pos = m_beobachterListe.begin(); pos!=m_beobachterListe.end(); ++pos)
18   {
19     (*pos)->aktualisiere();
20   }
21 }
```

- `meldeAn` fügt Beobachter der Liste hinzu
- `meldeAb` entfernt Beobachter aus Liste
- `benachrichtige()` ruft für alle Beobachter `aktualisiere()` auf

Beispiel C++: KonkretesSubjekt CAmpel

```
1  enum AmpelStatus {ROT, GRUEN};
2    //Aufzählungstyp: ROT = 0, GRUEN =1
3
4  class CAmpel: public CSubjekt
5  {
6    public:
7      CAmpel();
8      virtual ~CAmpel();
9      virtual AmpelStatus getStatus() const;
```

© R.-R. Grigat 2003-2019

```
10      virtual void setStatus(AmpelStatus);
11    private:
12      AmpelStatus m_status;
13  };
```

- CAmpel erbt von CSubjekt
- Zustand von CAmpel kann mit setStatus geändert werden
- setStatus muss benachrichtige aus CSubjekt aufrufen, damit alle Beobachter aktualisiert werden

Beispiel: KP15_4_enumStatus

Beispiel C++: Implementierung von KonkretesSubjekt

```
1  //campel.cpp
2  CAmpel::CAmpel():
3    m_status(ROT)
4  {}
5  CAmpel::~CAmpel()
6  {}
7  AmpelStatus CAmpel::getStatus() const
8  {
9    return m_status;
10 }
11 void CAmpel::setStatus(AmpelStatus status)
12 {
13   m_status = status;
14   benachrichtige();// bewirkt Aktualisierung aller Autos
15 }                  // Definiert in CSubjekt (Basisklasse von CAmpel)
```

- Zustand der Ampel ändert sich durch Aufruf von setStatus
- benachrichtige ruft aktualisiere für alle Objekte auf

Beispiel C++: KonkreterBeobachter CCabrio

```
1  class CCabrio : public CAuto
2  {
3    public:
4      CCabrio();
```

© R.-R. Grigat 2003-2019

```
5     virtual ~CCabrio();
6     virtual void aktualisiere();
7 };
```

```
1 CCabrio::CCabrio()
2 {}
3 CCabrio::~CCabrio()
4 {}
5 void CCabrio::aktualisiere()
6 {
7   cout << "Cabrio: neuer Ampelstatus: " ;
8   cout <<  m_ampel->getStatus() << endl;
9 }
```

- aktualisiere führt notwendige Operationen nach Zustandsänderung des KonkretenSubjekts durch
- Hier kann auch lokale Speicherung des neuen Zustands erfolgen

Beispiel C++: Anwendung

```
1  //main.cpp
2    CCabrio cabrio;
3    CTaxiAdapter taxi;
4      // Implementierung von aktualisiere() in CTaxiAdapter wie in CCabrio,
5      // aus Platzgründen hier weggelassen
6    CAmpel ampelAmWeg;
7
8    cabrio.starteBeobachtung(&ampelAmWeg);// fügt cabrio und taxi
9    taxi.starteBeobachtung(&ampelAmWeg);  // der Liste der Beobachter
10                                         // in CSubject hinzu
11   ampelAmWeg.setStatus(GRUEN);
12   //Ausgabe: Cabrio: neuer Ampelstatus: 1
13   //         Taxi: neuer Ampelstatus: 1
14
15   ampelAmWeg.setStatus(ROT);
16   //Ausgabe: Cabrio: neuer Ampelstatus: 0
17   //         Taxi: neuer Ampelstatus: 0
18
19   cabrio.endeBeobachtung(); // entfernt Cabrio aus der Liste
20   taxi.endeBeobachtung();   // der Beobachter in CSubject
```

© R.-R. Grigat 2003-2019

0.7.7 Versionsverwaltung

0.7.7.1 Versionsverwaltung

Aufgaben der Versionsverwaltung

- *Protokollierung* der Änderungen von Dokumenten oder Dateien (insbesondere Quelltexten, Büroanwendungen, Content-Management-Systemen)
- Erfassung mit Zeitstempel, Benutzerkennung und Kommentar (wer hat *wann was warum* geändert?)
- Vergisst nichts, ermöglicht Wiederherstellung *aller* gespeicherten Versionen selbst wenn Anwender diese als gelöscht markiert
- Archivierung der Entwicklungsversion (Trunk) und der an Kunden ausgelieferten Versionen (Branch, Tag) im Archiv (Repository), unabhängige Weiterentwicklung von Trunk und Branches möglich
- Merge, Zusammenführen von Versionen nach Bedarf
- Kurze Übertragungszeiten, überträgt nur Unterschiede von Dokumenten
- *Zentrale Versionsverwaltung* (SVN, CVS) führt auf Client-Serversysteme, Zugriff auf Server erfolgt mit Rechteverwaltung z.B. über Netzwerk, Versionsgeschichte nur im Repository vorhanden. Versionskette ist ein Vektor.
- *Verteilte Versionsverwaltung* (Mercurial) besitzt kein zentrales Repository mehr. Jedes Projektmitglied hat eigenes Repository und kann dieses mit jedem anderen Repository abgleichen. Die Versionsgeschichte ist verteilt, nicht zentral. Versionskette ist gerichteter azyklischer Graph.
- *Lock Modify Write* (pessimistische Versionsverwaltung) sperrt eine Datei zur Bearbeitung und gibt sie hinterher frei. Kein Zusammenführen von Änderungen erforderlich, aber nur ein Benutzer zur Zeit zulässig. Beispiele sind Revision Control System, Visual SourceSafe. Typisch für Binärdateien.
- *Copy Modify Write* (optimistische Versionsverwaltung) ermöglicht parallele Bearbeitung durch mehrere Nutzer. Änderungen werden hinterher automatisch oder manuell zusammengeführt (Merge).
- Siehe auch `http://de.wikipedia.org/wiki/Versionsverwaltung`

© R.-R. Grigat 2003-2019

0.7.7.2 Apache Subversion SVN

Merkmale und Befehle von SVN

- Nachfolger von CVS, ermöglicht Verschiebung und Kopie von Dokumenten ohne Versionsgeschichte zu verlieren, Revisionsnummern erfassen alle Änderungen sämtlicher Files eines Commit-Befehls.
- Unter Windows gute SVN-Unterstützung durch TortoiseSVN (`tortoisesvn.net`) und TortoiseMerge. Separate Installation von SVN *nicht* erforderlich.
- Verwaltung der Arbeitskopie im Folder .svn, diesen Folder also nicht löschen. Dort liegen Kopien der File-Originale und alle Änderungsschritte.
- Die folgenden Befehle dienen der SVN-Verwaltung durch den Anwender, SVN sperrt hierzu die Arbeitskopie.

Checkout Zuerst lokale Arbeitskopie des Dokuments anlegen (Verzeichnis auf dem lokalen Computer, wohin das gewählte Verzeichnis aus dem SVN kopiert wird). Der Befehl Checkout benötigt die Angabe beider Verzeichnisse und zusätzlich die Version, die aus dem SVN kopiert werden soll. Die neueste Version heißt Head-Revision.

Add Neue Files müssen explizit der SVN Versionsverwaltung unterstellt werden. Gespeichert werden neue Files (erst) mit Befehl *Commit*, sofern sie mit *Add* dem Repository bekannt gemacht wurden.

Ignore Temporäre Files und Zwischenversionen explizit von der Versionsverwaltung ausschließen.

Copy, Rename, Delete, Move
SVN Befehle verwenden und nicht Explorer-Befehle (sonst restauriert SVN bei Update z.B. gelöschte Files)! Gespeichert werden Änderungen erst mit *Commit*. Es sollten keine Verzeichnisse in der Arbeitskopie kopiert werden (linearer Aufwand O(n)) sondern nur auf dem Server (konstanter Aufwand O(1), da nur neuer Verweis angelegt wird, *billige* Kopie).

Update Aktualisierung der lokalen Arbeitskopie mit den zwischenzeitlichen Änderungen des Repositories.

Merge Beim *Update* können Konflikte zwischen lokalen Änderungen (*mine*) und Änderungen des Repositories (*theirs*) auftreten, diese müssen in der lokalen Kopie gelöst werden. Automatisch können nur *syntaktische* Konflikte gelöst werden, keine *semantischen* Konflikte.

© R.-R. Grigat 2003-2019

Commit Die lokalen geänderten Files werden als aktuelle Version ins Repository übernommen. Jedem Commit sollte ein *Update* und ggf. ein *Merge* unmittelbar vorausgehen. Es sollten nur kompilierfähige Sourcecodes als neue Version übergeben werden, keine Versionen mit Compiler- oder Linkerfehlern.

Branch/Tag Eine neuer Zweig (Branch) wird angelegt, z.B. für eine an Kunden ausgelieferte Version oder eine neue Entwicklungslinie. Tag markiert eine Version.

Switch Umschaltung zwischen Versionen (Trunk, Branches)

Check for modifications Statusabfrage der aktuellen Änderungen der lokalen Kopie.

Cleanup Operationen werden von SVN zunächst in Protokolldatei geschrieben und von dort ausgeführt. Cleanup führt etwaige Operationen aus der Protokolldatei aus (soweit nach Rechnerabsturz oder Prozessterminierung noch vorhanden) und entfernt danach Sperren der Arbeitskopie.

0.7.7.3 Mercurial

Eigenschaften von Mercurial

- Plattform-unabhängiges verteiltes Versionskontrollsystem zur Software-Entwicklung
- Anwender arbeitet auf lokaler Kopie des Repositories, kann sämtliche Repository-Operationen auf der lokalen Kopie ausführen
- Synchronisierung zwischen Kopien des Repositories oder mit dem Server-Repository
- Plattformunabhängig, grafische Unterstützung für Windows z.B. durch TortoiseHg